

Introduction to Ontology Engineering

A Two-Week Module for Artificial Intelligence Capstone

Chun-yien Chang, PhD Candidate

Department of Computer Science, NYCU

ccy@hptp.org

May 2026

Acknowledgment

Why this module matters

I would first like to thank Professor Yi-Ting for making ontology engineering part of this AI Capstone.

Why this is significant

At a time when computer science is increasingly organized around data, models, and scalable AI systems, it is especially meaningful to reserve space for a discipline concerned with *what our systems mean, what distinctions they preserve, and what kinds of knowledge they make explicit.*

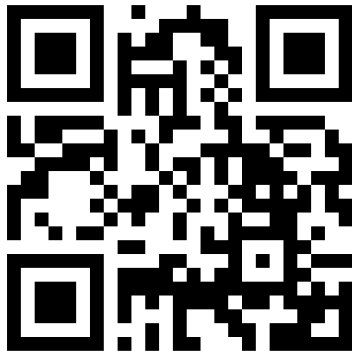
Before We Begin

A quick interaction check with Vevox

- ▶ We will use **Vevox** for several interactive questions during this module.
- ▶ These responses will help me assess how well the new unit is being absorbed.
- ▶ Please join the session using the QR code on the right.

Registration format

- ▶ **First name:** Student ID
- ▶ **Last name:** Your name



Which of the following is an important AI textbook for this course?

- A. *Foundations of Autonomous Intelligence*
- B. *Artificial Minds and Machine Reasoning*
- C. *Artificial Intelligence: A Modern Approach*
- D. *The Computational Theory of Smart Systems*

Interaction check

This is our first Vevox question. Please try to answer it correctly.

This slide is intentionally hidden.

The content is reserved for the in-class session.

A Necessary Scope Decision

Two weeks require a direct route

- ▶ Ontology engineering has important foundations in logic, philosophy, and knowledge representation.
- ▶ In a fuller course, we would spend time on these foundations before moving into ontology design.
- ▶ In this module, however, we have only **two weeks**.

Instructional choice

We will therefore move directly into the *engineering problem*: how to make domain meaning explicit, structured, and usable in AI and Physical AI systems.

What We Will Temporarily Set Aside

Useful foundations, but not today's route

Topics we will not develop in depth

- ▶ Logic foundations.
- ▶ Philosophical origins of ontology.
- ▶ The broader history of knowledge representation.
- ▶ Formal semantic theory beyond what is immediately needed.

Implication

These foundations remain important. They are being bracketed here so that we can focus on ontology engineering as a practical response to a concrete AI-era problem.

Recommended starting points. For the logical background, see the discussion of first-order logic in [Russell, S. J., & Norvig, P. \(2021\). *Artificial intelligence: A modern approach* \(4th ed.\). Pearson.](#) For a broader introduction to ontology engineering, see Chapter 1 of [Keet, C. M. \(2025\). *An introduction to ontology engineering* \(2nd ed.\). College Publications.](#)

Defining Ontology

From prior familiarity to an engineering definition

Have you encountered the term “ontology” before?

Have you heard or seen the term “ontology” before?

- A. Yes, in a computer science context.
- B. Yes, but not in a computer science context.
- C. No.

Purpose of this poll

This question helps establish our starting point before defining ontology more carefully.

A Classic Starting Point

Gruber's definition of ontology

An *ontology* is an explicit specification of a conceptualization. The term is borrowed from philosophy, where an ontology is a systematic account of Existence. For knowledge-based systems, what “exists” is exactly that which can be represented.

We will use this definition as our first entrance into ontology engineering.

Primary source: Gruber, T. R. (1993). A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2), 199–220.
<https://doi.org/10.1006/knac.1993.1008>

What Does Gruber's Definition Mean?

A practical reading

Conceptualization

How we understand a domain: its main things and their relations.

Specification

We write that understanding down in a structured way.

Explicit

The meaning is not only in our heads. Others can inspect it.

Simple takeaway

An ontology makes a domain understanding explicit and structured.

Primary source: Gruber, T. R. (1993). A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2), 199–220.
<https://doi.org/10.1006/knac.1993.1008>

Why Does This Become an Engineering Problem?

Shared meaning does not happen automatically

In practice

- ▶ Different people use the same word differently.
- ▶ Different systems store related information differently.
- ▶ Important assumptions are often hidden.

Ontology engineering asks

- ▶ What should we represent?
- ▶ How should we define it?
- ▶ How can others reuse it correctly?

Transition

This is the point where ontology becomes an engineering task.

A Familiar Data Table

Student records in a university system

ID	Name	Unit	Email
S001	Amy Chen	NYCU Department of Computer Science	amy.cs@example.edu
S002	Brian Lee	NYCU Institute of Music	brian.music@example.edu
S003	Claire Wang	NYCU Department of Computer Science	claire.cs@example.edu

At first glance

This looks like a simple Student table: one row for one person, with an ID, a name, an academic unit, and an email address.

What the Table Does Not Say

Important meaning is still implicit

What we can see

- ▶ A table named Student.
- ▶ Three rows of records.
- ▶ Columns for ID, name, unit, and email.

What we silently assume

- ▶ Each row represents a student.
- ▶ The person is currently enrolled.
- ▶ The unit is an academic organization.
- ▶ The email belongs to that person.

Key point

The table stores values, but its intended meaning is only partially stated.

Which meaning is still implicit in the Student table?

Please answer in Vevox

Think about the difference between what the table *stores* and what we *assume it means*.

This slide is intentionally hidden.

The content is reserved for the in-class session.

A Graph Is Built from Simple Statements

One edge, one basic claim



Subject	ex:AmyChen
Predicate	schema:affiliation
Object	ex:CSDept

RDF idea

RDF represents knowledge through *subject–predicate–object* statements, called *triples*.

The Same Statement in RDF/XML

Just a quick look

```
<rdf:RDF
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:schema="https://schema.org/"
  xmlns:ex="https://example.org/nycu/">

  <rdf:Description rdf:about="https://example.org/nycu/AmyChen">
    <schema:affiliation
      rdf:resource="https://example.org/nycu/CSDept"/>
  </rdf:Description>

</rdf:RDF>
```

This RDF/XML (*Resource Description Framework in XML syntax*) fragment expresses the same relation as `ex:AmyChen schema:affiliation ex:CSDept` .

What Is RDF?

A simple graph-based data model

RDF

RDF stands for *Resource Description Framework*.

- ▶ It is a way to describe things and their relations.
- ▶ It represents information as a graph.

Important distinction

RDF is a *data model*, not a specific file format.

- ▶ The graph is the main idea.
- ▶ Later, we will use Turtle to write it as text.

Reading an RDF Triple

A statement about the world

`ex:AmyChen` `schema:affiliation` `ex:CSDept`

Subject

The thing we are
describing.

`ex:AmyChen`

Predicate

The relation we state.
`schema:affiliation`

Object

The thing connected to it.

`ex:CSDept`

Plain-English reading

Amy Chen is affiliated with the Department of Computer Science.

In the triple below, which part is the predicate?

ex:AmyChen schema:affiliation ex:CSDept

Please answer in Vevox

Identify the part that expresses the relation between the subject and the object.

This slide is intentionally hidden.

The content is reserved for the in-class session.

Returning to the Student Example

The graph makes more meaning available

In the table

- ▶ “Department of Computer Science” appears as a cell value.
- ▶ “Institute of Music” appears as another cell value.
- ▶ Their relation to the university is not stated.

In the graph

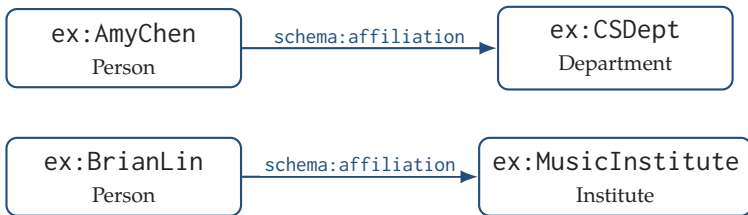
- ▶ Academic units can become entities.
- ▶ Their organizational relations can be stated.
- ▶ People can be linked to those units.

Point

The graph does not only store values. It gives those values an explicit relational structure.

Adding People to the Graph

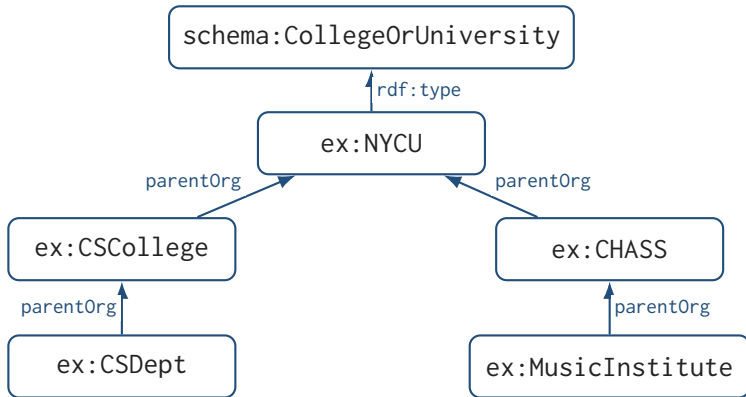
Affiliation becomes an explicit relation



The graph states what the table only suggests: a person is affiliated with a specific organization.

The Same Meaning as a Graph

Organizations become explicit entities



A student table may mention units. An RDF graph can also state how those units are organized.

The Organizational Graph in RDF/XML

Just a quick look

```
<rdf:RDF
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:schema="https://schema.org/"
  xmlns:ex="https://example.org/nycu/">

  <rdf:Description rdf:about="https://example.org/nycu/NYCU">
    <rdf:type rdf:resource="https://schema.org/CollegeOrUniversity"/>
  </rdf:Description>
  <rdf:Description rdf:about="https://example.org/nycu/CSCollege">
    <schema:parentOrganization rdf:resource="https://example.org/nycu/NYCU"/>
  </rdf:Description>
  <rdf:Description rdf:about="https://example.org/nycu/CSDept">
    <schema:parentOrganization rdf:resource="https://example.org/nycu/CSCollege"/>
  </rdf:Description>

</rdf:RDF>
```

This RDF/XML fragment expresses part of the organizational graph: NYCU is a university, the CS College belongs to NYCU, and the CS Department belongs to the CS College.

What Can We Now Say?

A richer view of the same small example

- ▶ Amy Chen is affiliated with the Department of Computer Science.
- ▶ The Department of Computer Science belongs to the College of Computer Science.
- ▶ The College of Computer Science belongs to NYCU.
- ▶ The Institute of Music also belongs to NYCU.

Why this matters

The original table did not clearly represent these relations. An RDF graph can make them available for later query, linking, and inference.

From RDF Graph to Turtle

A readable textual syntax

RDF graph

The model:

- ▶ things as nodes,
- ▶ relations as edges,
- ▶ statements as triples.

Turtle

A compact way to write RDF graphs as text.

- ▶ Easier to read than RDF/XML.
- ▶ Convenient for teaching and editing.

Scope note

RDF can also be written in RDF/XML. Because this module is short, we will use Turtle and not develop the XML foundation in detail.

One RDF Triple in Turtle

The graph statement written as text

```
ex:AmyChen schema:affiliation ex:CSDept .
```

Reading

This is the Turtle form of the same RDF triple: Amy Chen is affiliated with the Department of Computer Science.

We will explain names such as `ex:` and `schema:` shortly.

Several Statements about One Person

Turtle groups related triples

```
ex:AmyChen
  a schema:Person ;
  schema:name "Amy Chen" ;
  schema:affiliation ex:CSDept .
```

Reading

The same subject can appear in several triples: Amy is a person, has a name, and is affiliated with the CS department.

What does Turtle help us do in this example?

Please answer in Vevox

Recall the distinction: RDF is the graph-based data model; Turtle is the readable textual syntax we use to write it.

This slide is intentionally hidden.

The content is reserved for the in-class session.

When the Meaning Is Not Fixed

A table can work while its semantics drift

The intended meaning

- ▶ Every row is a student.
- ▶ The table supports student-related queries.
- ▶ Later analysis assumes this category is stable.

What may happen in practice

- ▶ A visiting trainee is added for system access.
- ▶ A temporary assistant is inserted for convenience.
- ▶ A graduated student remains in the table.

Consequence

If Student is not explicitly defined, the table may gradually contain people who are operationally convenient to store there, but who are not students in the intended sense.

Why This Is Already Richer Than the Table

The same values now carry explicit meaning

Aspect	In the table	In the RDF description
Amy Chen	A row value	An explicitly described person
Department	Text in a cell	A linked organization entity
Affiliation	Only implied by table layout	A stated relation between entities

Implication

The table records values. The RDF description states what those values mean and how they are connected.

What Are These Compact Names?

Prefixes stand for namespaces

Compact form

schema:Person
schema:affiliation
ex:AmyChen

What they do

A prefix shortens a longer identifier, so we do not have to repeat the full web address every time.

Key point

Namespaces make vocabularies extendable: we can reuse terms from schema.org and add our own local terms when needed.

Why Extensibility Matters

Shared vocabularies do not cover every local need

schema.org helps us state

- ▶ Person
- ▶ College or university
- ▶ Organization
- ▶ Affiliation
- ▶ Parent organization

But we may still need

- ▶ Current student status
- ▶ Degree program
- ▶ Enrollment period
- ▶ Department-specific rules

Inference

Ontology representation becomes powerful because shared vocabularies can be reused and local meanings can be added without discarding the existing structure.

Declaring the Namespaces

The part we omitted earlier

```
@prefix schema: <https://schema.org/> .  
@prefix ex:      <https://example.org/nycu/> .
```

schema:

Points to the shared schema.org
vocabulary.

ex:

Points to our local example vocabulary
and entities.

From Prefix to Full URI

What does `schema:affiliation` really stand for?

`schema:affiliation`



`<https://schema.org/affiliation>`

Reading

The prefix `schema:` is a shorthand for `https://schema.org/`. So `schema:affiliation` expands into a complete identifier.

URI: Uniform Resource Identifier. In RDF, a URI is a web-style identifier used to name a resource, such as an entity, a class, or a relation. In RDF 1.1, you may also see the term **IRI** (*Internationalized Resource Identifier*). It is a generalization of URI that allows non-ASCII characters; see the cited source for details.

Primary source: World Wide Web Consortium. (2014b). Rdf 1.1 primer [W3C Note, 24 June 2014]. <https://www.w3.org/TR/rdf11-primer/>

What Is a URI?

A resource identifier on the Web

```
URI = scheme ":" ["/" authority] path ["?" query] ["#" fragment]
```

Part	Example	Role
Scheme	https	The access or identification scheme.
Authority	schema.org	The domain responsible for this identifier space.
Path	/affiliation	The specific term being identified.

Why does it look like a URL? A URI may look like the URLs (Uniform Resource Locators) we use every day, but its primary role is to *identify* a resource, not necessarily to retrieve a web page. A URL is the familiar subset of URI that also provides a way to locate a resource on the World Wide Web.

Primary source: Berners-Lee, T., Fielding, R. T., & Masinter, L. (2005). Uniform resource identifier (uri): Generic syntax. <https://datatracker.ietf.org/doc/html/rfc3986>

The Identifier Leads to a Definition

A shared vocabulary is inspectable

The URI

`https://schema.org/affiliation`

Its role in our triple

`ex:AmyChen`
`schema:affiliation`
`ex:CSDept`

Point

The relation in our graph is not just a local label. It reuses a term whose meaning is publicly documented.

Schema.org states that:

- ▶ affiliation is a property.
- ▶ It is used on Person.
- ▶ Its expected value is Organization.

Why is `schema:affiliation` more than just a convenient label?

Please answer in Vevox

Think about what we just saw: a compact name, a full URI, and a publicly documented definition.

This slide is intentionally hidden.

The content is reserved for the in-class session.

Why Does This Look Familiar?

From everyday URLs to Semantic Web identifiers

In everyday Web use

- ▶ We type or click URLs.
- ▶ They lead us to web pages, files, or services.
- ▶ Example: a page on `schema.org`.

In RDF and ontology

- ▶ We use URIs to identify resources.
- ▶ The resource may be a person, a concept, or a relation.
- ▶ URIs can also function as URLs, opening useful documentation pages.

Transition

This is the bridge to the *Semantic Web*: the Web is not only used to link documents, but also to identify and connect meanings.

Primary source: World Wide Web Consortium. (2014a). Rdf 1.1 concepts and abstract syntax [W3C Recommendation, 25 February 2014].
<https://www.w3.org/TR/rdf11-concepts/>

A Roadmap of What We Just Did

From implicit table meaning to Web-style semantic identification

Step	What changed
Database table	Values are stored, but much of their intended meaning remains implicit.
Semantic graph	Entities and relations become explicit rather than being hidden in table layout.
RDF triples	Each graph relation can be stated as a subject–predicate–object statement.
Turtle syntax	Those RDF statements can be written in a compact, human-readable textual form.
Namespaces and URIs	Terms such as <code>schema:affiliation</code> expand into globally identifiable Web-style resources.